COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010-DSA	**SECTION:**	CpE21S1
DATE PERFORMED:	8-4-2026	**DATE SUBMITTED:**	8-11-2026
NAME:	Mauro, Ryan Lewis B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 [Queues]

1. PROCEDURE OUTPUT

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> r);

int main() {

    std::queue<char> q;

    q.push('c');
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('1');
    q.push('0');
```

```cpp
// Display
std::cout << "Initial Queue:";
display(q);


// empty()
std::cout<<"Is the queue empty? "<<q.empty() <<std::endl;

// size()
std::cout << "Queue size: " << q.size() << std::endl;


// front()
std::cout << "Front element: " << q.front() << std::endl;


// back()
std::cout << "Back element: " << q.back() << std::endl;


// pop()
std::cout << "\nRemoving the front element...\n";
q.pop();


// display
std::cout << "Queue after pop:";
display(q);
```

```
    // push another element
    std::cout << "\nAdding element 'X'...\n";
    q.push('X');

    // display
    std::cout << "Queue after push:";
    display(q);


    // Delete the queue
    while (!q.empty()) {
        q.pop();
}

std::cout << "Queue deleted.\n";
```

```
    // Display after deleting
    std::cout << "Queue after delete:";
    display(q);

    std::cout << "Queue size: " << q.size() << std::endl;

    return 0;
}


void display(std::queue<char> r) {
    // Create a copy of the queue
    std::queue<char> c = r;


    while (!c.empty()) {
        std::cout << " " << c.front();
        c.pop();
    }
    std::cout << std::endl;
}
```

```
Initial Queue: c p e 0 1 0
Is the queue empty? 0
Queue size: 6
Front element: c
Back element: 0

Removing the front element...
Queue after pop: p e 0 1 0

Adding element 'X'...
Queue after push: p e 0 1 0 X
Queue deleted.
Queue after delete:
Queue size: 0
```

Analysis: This program demonstrates the basic operations of the C++ queue container. It begins by inserting several characters into the queue using push(), following the First In, First Out (FIFO) principle. The program then checks whether the queue is empty, displays its size, and retrieves the front and back elements without removing them. Next, the pop() function removes the front element, showing that the first item inserted is also the first one removed. Another element is added using push(), and the updated queue is displayed. Finally, all remaining elements are removed using a while loop with pop(), effectively deleting or clearing the queue. The program confirms that the queue is empty by displaying its contents and checking its size, demonstrating the proper use of common queue operations in C++.

Table 5.1 - For ILOA

```cpp
#include <iostream>
#include "queuell.h"


int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;
    // Enqueue elements
    enqueue(&front, &back, 'R');
    enqueue(&front, &back, 'Y');
    enqueue(&front, &back, 'A');
    enqueue(&front, &back, 'N');

    // Display queue
    std::cout << "Queue: ";
    display(front);

    // Return the front element
    std::cout << "Front element: " << getFront(front) << std::endl;

    // Dequeue one element
    dequeue(&front, &back);
    std::cout << "\nAfter dequeue:" << std::endl;
    std::cout << "Queue: ";
    display(front);
    std::cout << "Front element: " << getFront(front) << std::endl;

    return 0;
}
```

```
Queue: R Y A N
Front element: R

After dequeue:
Queue: Y A N
Front element: Y
```

Analysis: The main.cpp file demonstrates how the queue functions are used. It starts by creating an empty queue with front and back pointers initialized to nullptr. Several characters are inserted into the queue using enqueue(), and the contents are displayed to verify the insertion order. The program then retrieves the front element using getFront(), removes the first element with dequeue(), and displays the updated queue. The output confirms that the queue correctly follows the FIFO (First In, First Out) behavior, where the first element inserted is also the first element removed. This file serves as a simple test program to verify that all queue operations work correctly.

Table 5.2 - For ILOB (.cpp file)

```
#ifndef QUEUELL_H
#define QUEUELL_H

template <typename T>
class qNode{
public:
    T data;
    qNode<T> *next;
};

// Create a new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}
```

```
// Insert an element at the back of the queue
template <typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData){

    qNode<T>* newNode = new_node(newData);

    // Queue is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        *frontPtr = newNode;
        *backPtr = newNode;
        return;
    }

    // Queue is not empty
    (*backPtr)->next = newNode;
    *backPtr = newNode;
}
```

```cpp
// Remove the front element
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){


    // Check if queue is empty
    if(*frontPtr == nullptr){
        std::cout << "Queue is empty!" << std::endl;
        return;
    }

    qNode<T>* deleteNode = *frontPtr;

    // Queue has only one node
    if((*frontPtr)->next == nullptr){
        *frontPtr = nullptr;
        *backPtr = nullptr;
        delete deleteNode;
        return;
    }

    // Queue has more than one node
    *frontPtr = deleteNode->next;
    delete deleteNode;
}
```

```cpp
// Display all elements
template <typename T>
void display(qNode<T>* front){
    qNode<T>* temp = front;

    while(temp != nullptr){
        std::cout << temp->data << " ";
        temp = temp->next;
    }
    std::cout << std::endl;
}

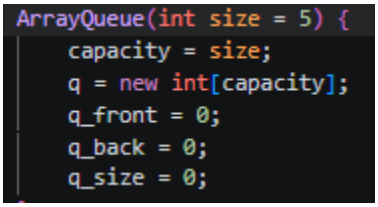
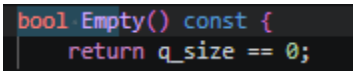
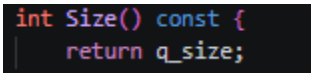
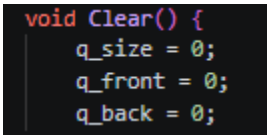
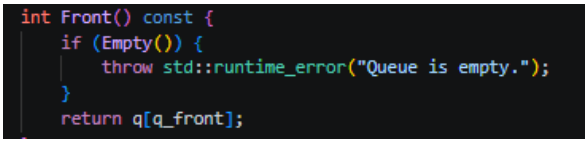
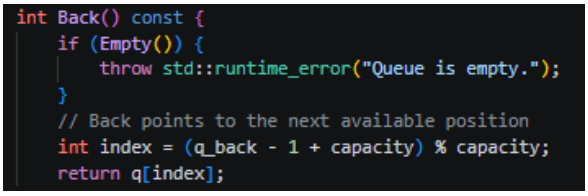
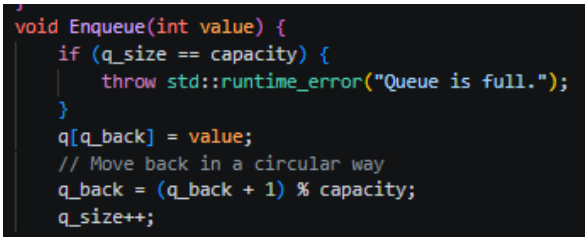
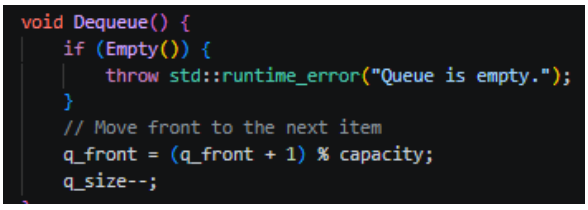
// Return the front element
template <typename T>
T getFront(qNode<T>* front){

    if(front == nullptr){
     std::cout<<"Queue is empty!";
    }
    return front->data;
}

#endif
```

Analysis: The header file contains the implementation of a queue using a linked list and C++ templates, allowing it to store different data types. It defines the qNode class to represent each node, where each node contains data and a pointer to the next node. The new_node() function dynamically creates a new node, while enqueue() inserts a new node at the back of the queue. The dequeue() function removes the front node, ensuring the queue follows the First In, First Out (FIFO) principle. The display() function traverses the linked list and prints all elements in order, and getFront() returns the value of the front node without removing it. Overall, the header file provides all the essential queue operations through reusable template functions.

Table 5.3 - For ILOB (.h file)

Operation	Sreenshot
Queue() (Constructor)	
Empty()	
Size()	
Clear()	
Front()	
Back()	
Enqueue()	
Dequeue()	

```
ArrayQueue(int size = 5) {
    capacity = size;
    q = new int[capacity];
    q_front = 0;
    q_back = 0;
    q_size = 0;
```

```
bool Empty() const {
    return q_size == 0;
```

```
int Size() const {
    return q_size;
```

```
void Clear() {
    q_size = 0;
    q_front = 0;
    q_back = 0;
```

```
int Front() const {
    if (Empty()) {
        throw std::runtime_error("Queue is empty.");
    }
    return q[q_front];
```

```
int Back() const {
    if (Empty()) {
        throw std::runtime_error("Queue is empty.");
    }
    // Back points to the next available position
    int index = (q_back - 1 + capacity) % capacity;
    return q[index];
```

```
void Enqueue(int value) {
    if (q_size == capacity) {
        throw std::runtime_error("Queue is full.");
    }
    q[q_back] = value;
    // Move back in a circular way
    q_back = (q_back + 1) % capacity;
    q_size++;
```

```
void Dequeue() {
    if (Empty()) {
        throw std::runtime_error("Queue is empty.");
    }
    // Move front to the next item
    q_front = (q_front + 1) % capacity;
    q_size--;
```

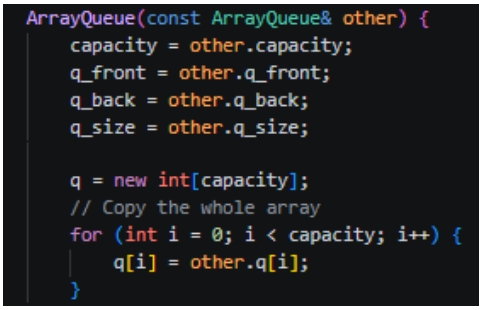
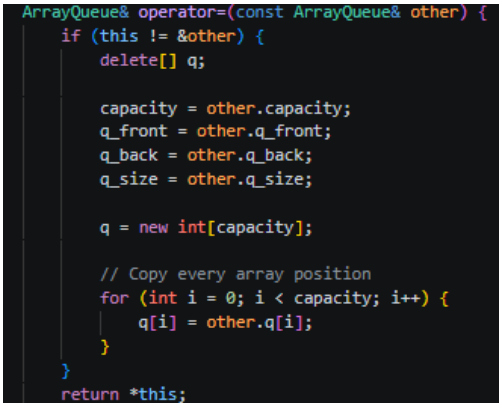
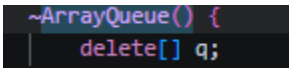
Copy Constructor	
Copy Assignment Operator	
Destructor	

Table 5.3 - For ILOB 2

The **ArrayQueue** class uses a dynamically allocated array to store the queue elements. The **constructor** creates the array and sets the queue values to their starting positions. **Enqueue()** adds an element to the back, while **Dequeue()** removes the element from the front. **Front()** and **Back()** are used to check the first and last elements without removing them. **Empty()** checks if the queue has no elements, while **Size()** returns the current number of elements. **Clear()** resets the queue to an empty state. The **copy constructor** creates a separate queue with the same contents as another queue. The **copy assignment operator** replaces one queue's contents with another queue's contents.The **destructor** uses **delete[]** to release the dynamically allocated memory when the queue is no longer needed. The circular array also allows the queue to reuse available spaces instead of shifting the elements.

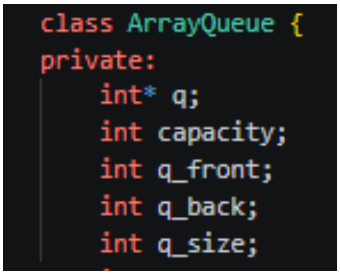


Figure 5.1 - For ILO B 2

This screenshot shows the **ArrayQueue** class and its private data members. The **q pointer** is used to create the array dynamically, while capacity, **q_front, q_back, and q_size** keep track of the queue's capacity, front position, back position, and number of elements.

```cpp
int main() {
    ArrayQueue queue(5);
    // Add some values
    queue.Enqueue(10);
    queue.Enqueue(20);
    queue.Enqueue(30);

    std::cout << "Initial queue: ";
    queue.Display();

    std::cout << "Front: " << queue.Front() << "\n";
    std::cout << "Back: " << queue.Back() << "\n";
    std::cout << "Size: " << queue.Size() << "\n";

    // Remove the front item
    queue.Dequeue();

    std::cout << "\nAfter dequeue: ";
    queue.Display();

    // Add another value
    queue.Enqueue(40);
    queue.Enqueue(50);

    std::cout << "After adding 40 and 50: ";
    queue.Display();

    // Demonstrate the circular array
    queue.Dequeue();
    queue.Enqueue(60);

    std::cout << "After dequeue and adding 60: ";
    queue.Display();

    // Test copy constructor
    ArrayQueue copiedQueue(queue);

    std::cout << "\nCopied queue: ";
    copiedQueue.Display();

    // Test copy assignment
    ArrayQueue assignedQueue;
    assignedQueue = queue;

    std::cout << "Assigned queue: ";
    assignedQueue.Display();

    // Test Clear
    queue.Clear();

    std::cout << "\nAfter Clear(): ";
    queue.Display();
```

Figure 5.2 - For ILO B 2

This screenshot shows the **main()** function where I tested the queue operations. I used **Enqueue(), Dequeue(), Front(), Back(), Size(), and Clear()** to make sure the queue works properly and produces the expected results.

```
Initial queue: 10 20 30
Front: 10
Back: 30
Size: 3

After dequeue: 20 30
After adding 40 and 50: 20 30 40 50
After dequeue and adding 60: 30 40 50 60

Copied queue: 30 40 50 60
Assigned queue: 30 40 50 60

After Clear(): Queue is empty.
```

Figure 5.2 - Output for ILO B 2

This screenshot shows the actual output of the program after running the queue operations. It confirms that the queue was able to add, remove, access, copy, and clear elements correctly. The output also shows that the circular queue works as expected when new elements are added after a dequeue.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
class Job {
private:
    int jobID;
    std::string userName;
    int pages;
public:
    // Constructor
    Job(int id = 0, std::string name = "", int pageCount = 0) {
        jobID = id;
        userName = name;
        pages = pageCount;
    }

    // Display job information
    void Display() const {
        std::cout << "Job ID: " << jobID
                  << " | User: " << userName
                  << " | Pages: " << pages << "\n";
    }
```

Figure 5.3 - For ILO C

This screenshot shows the **Job** class, which stores the important information for each printing job. It includes the job ID, the user's name, and the number of pages to be printed.

```
class Printer {
private:
    Job* jobs;
    int capacity;
    int front;
    int back;
    int size;
public:
    // Constructor
    Printer(int maxJobs = 10) {
        capacity = maxJobs;
        jobs = new Job[capacity];
        front = 0;
        back = 0;
        size = 0;
    }
    // Destructor
    ~Printer() {
        delete[] jobs;
    }
    // Check if the printer queue is empty
    bool Empty() const {
        return size == 0;
    }
    // Add a new print job
    void AddJob(const Job& job) {
        if (size == capacity) {
            std::cout << "Printer queue is full.\n";
            return;
        }
        jobs[back] = job;
        back = (back + 1) % capacity;
        size++;
        std::cout << "Job added to printer queue.\n";
    }

    // Process the first job in the queue
    void ProcessJob() {
        if (Empty()) {
            std::cout << "No jobs to process.\n";
            return;
        }
        std::cout << "Processing: ";
        jobs[front].Display();
        front = (front + 1) % capacity;
        size--;
    }

    // Process all remaining jobs
    void ProcessAllJobs() {
        std::cout << "\nProcessing all remaining jobs:\n";
        while (!Empty()) {
            ProcessJob();
        }
    }
    // Display all pending jobs
    void DisplayQueue() const {
        if (Empty()) {
            std::cout << "Printer queue is empty.\n";
            return;
        }
        std::cout << "Pending printer jobs:\n";
        for (int i = 0; i < size; i++) {
            int index = (front + i) % capacity;
            jobs[index].Display();
        }
    }
```

Figure 5.4 - For ILO C

This screenshot shows the **Printer** class, which is responsible for managing all the pending printing jobs. The class uses an array of Job objects to store the jobs and keeps track of the front, back, size, and capacity of the queue. It also contains functions such as **AddJob(), ProcessJob**(), and **ProcessAllJobs()** to control how jobs are added and processed. This class allows the printer to organize multiple print requests and handle them in an orderly way without using the STL queue.

```cpp
// Add a new print job
void AddJob(const Job& job) {
    if (size == capacity) {
        std::cout << "Printer queue is full.\n";
        return;
    }
    jobs[back] = job;
    back = (back + 1) % capacity;
    size++;
    std::cout << "Job added to printer queue.\n";
}

// Process the first job in the queue
void ProcessJob() {
    if (Empty()) {
        std::cout << "No jobs to process.\n";
        return;
    }
    std::cout << "Processing: ";
    jobs[front].Display();
    front = (front + 1) % capacity;
    size--;
}
```

Figure 5.5 - For ILO C

This screenshot shows how the queue follows the **First In, First out rule**. New jobs are added at the back of the queue, while the job at the front is processed first.

```cpp
Job job1(101, "Ryan", 5);
Job job2(102, "Lewis", 10);
Job job3(103, "Bibiano", 3);
Job job4(104, "Mauro", 8);

printer.AddJob(job1);
printer.AddJob(job2);
printer.AddJob(job3);
printer.AddJob(job4);

// Display pending jobs
std::cout << "\n--- Current Printer Queue ---\n";
printer.DisplayQueue();

// Process jobs one by one
std::cout << "\n--- Processing Jobs ---\n";

printer.ProcessJob();
printer.ProcessJob();
```

Figure 5.6 - For ILO C

This screenshot shows the simulation where multiple users submit their printing jobs. The printer then **processes the jobs one at a time** according to the order they were added.

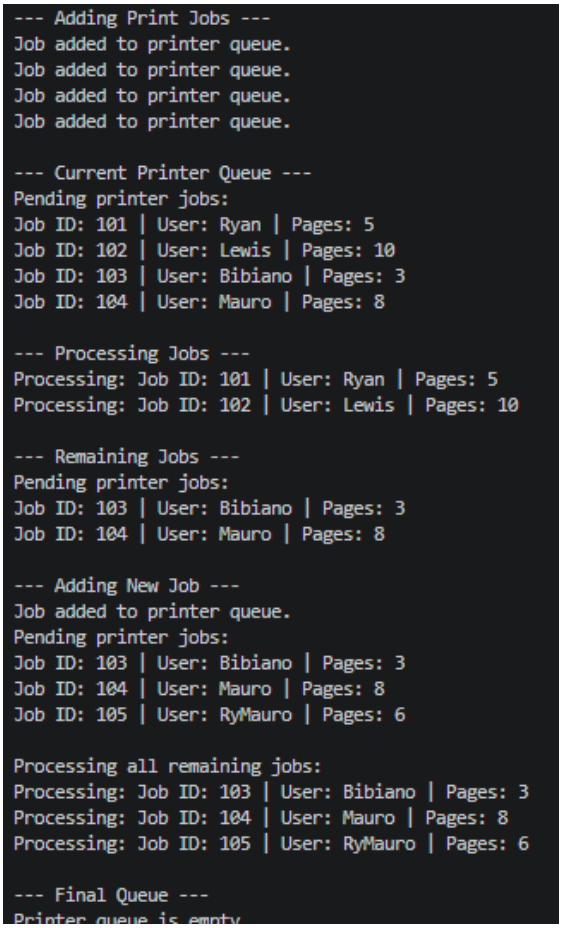


Figure 5.7 - Output for ILO C

This screenshot shows the actual output of the shared printer simulation after the program was executed. It demonstrates that five different users, Ryan, Lewis, Bibiano, Mauro, and RyMauro, submitted their printing jobs to the shared printer. The jobs were placed into the queue in the order they were added, with Ryan's job being first, followed by Lewis, Bibiano, Mauro, and RyMauro. The printer processes the jobs one at a time starting from the front of the queue, following the First in, First out strategy. As each job is completed, it is removed from the queue and the next pending job is processed. The output confirms that all five users' jobs were handled in the correct order and that the queue became empty after all printing jobs were completed.

Defend your choice of internal representation: Why did you use arrays or linked lists?

I chose an array implementation for the printer queue because it is simple and suitable for this simulation. The array stores the Job objects in an organized way, while the front, back, and size variables keep track of the current queue. I also used a circular array so that spaces from completed jobs can be reused instead of being wasted. This implementation supports the First in, First out strategy because new jobs are placed at the back and the first job is processed from the front. I chose an array instead of a linked list because the printer simulation has a set maximum capacity, making the array easier to manage and understand for this activity.

Conclusion for this activity:

In conclusion, this activity helped me understand how an array-based queue works and how its different operations are connected. I learned how to add and remove elements using Enqueue() and Dequeue(), check the first and last elements using Front() and Back(), and manage the queue using Empty(), Size(), and Clear(). I also learned the importance of the copy constructor, copy assignment operator, and destructor when working with dynamically allocated memory. This activity improved my understanding of queue implementation, circular arrays, and proper memory management in C++.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.